UNIVERSIDAD TECNOLÓGICA METROPOLITANA

Carrera: Ingeniería en Tecnologías de la Información e Innovación Digital
Asignatura: Estructura de Datos

PRÁCTICA DE LA UNIDAD II

Estructuras de datos básicas: listas, pilas y colas

Versión web desarrollada en C#

Docente:
Ing. Mirian Magaly Canché Caamal

Integrantes:

- Santiago Nicolás De la Mora Núñez
- Jesus Joel Canche Chac
- Roger Alejandro Aguilar Nuñez
- Victor Manuel Almendarez García

Grupo: 4.° A
Fecha de entrega: 23 de octubre de 2026

L1. Productos disponibles y retirados

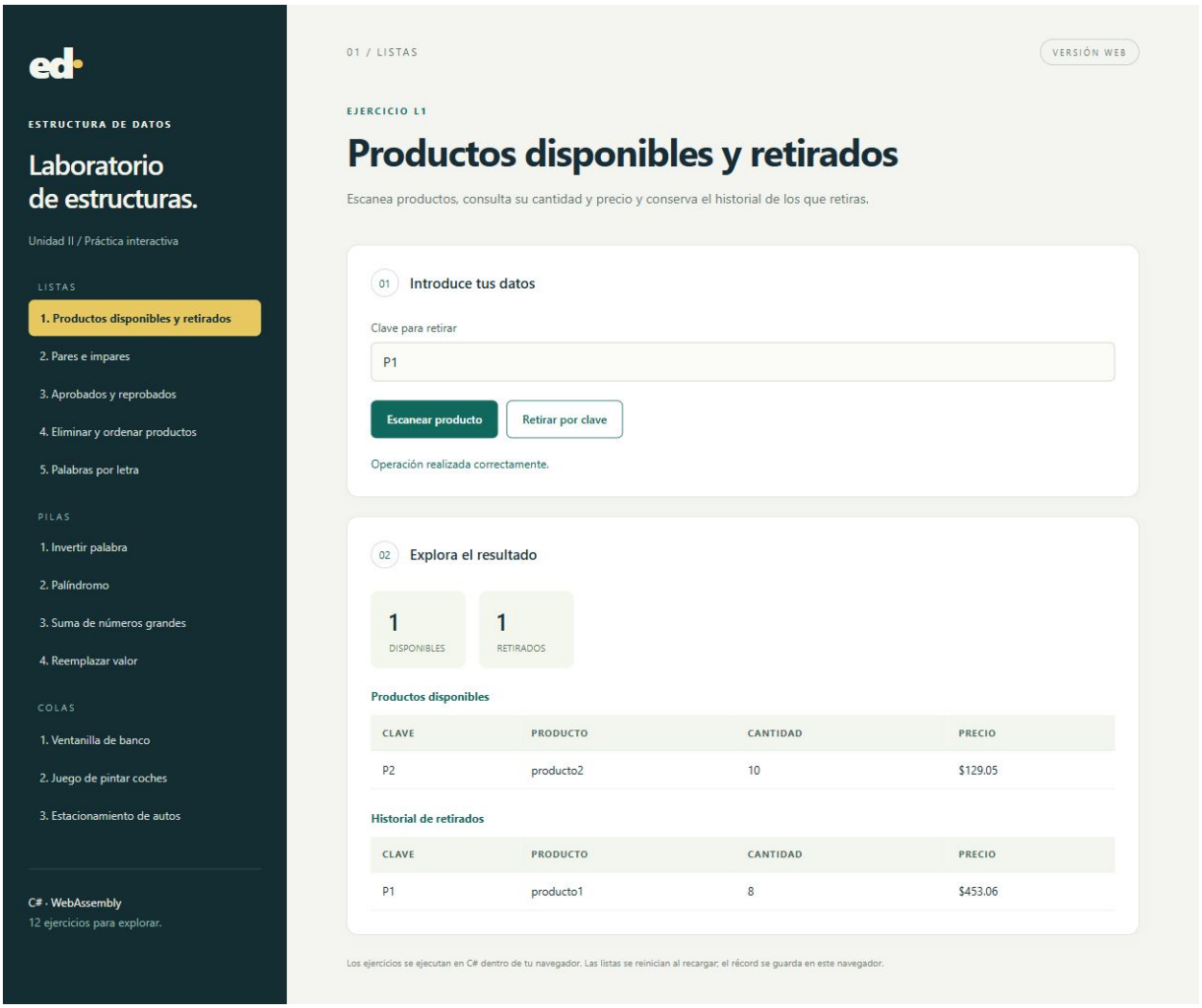
Problema: Administrar el almacén de productos escaneados y conservar los retirados.

Entradas: Clave del producto a retirar; la cantidad y el precio se generan automáticamente.

Proceso: Escanear crea un objeto Producto y lo agrega a List<Producto>. Retirar busca por clave y mueve el objeto a otra lista.

Salidas: Productos disponibles y retirados, con cantidades, precios y número de elementos.

Funcionamiento y evidencia: Pulsa Escanear producto dos veces; escribe P1 y pulsa Retirar producto. La imagen muestra el producto1 retirado y el producto2 disponible. Una clave inexistente produce un mensaje sin perder datos.



L2. Pares e impares

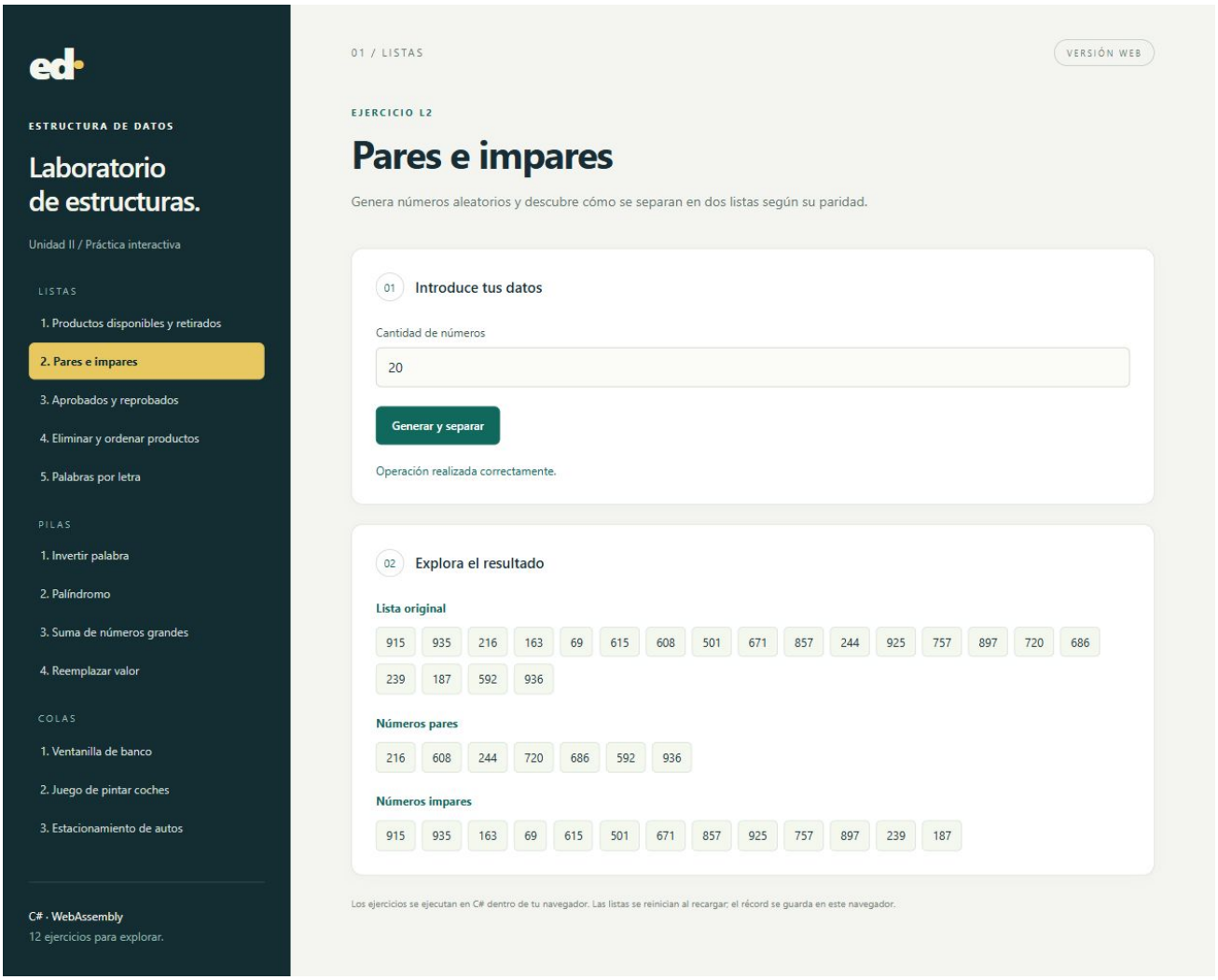
Problema: Separar una lista de números aleatorios en listas por paridad.

Entradas: Cantidad entera entre 1 y 500.

Proceso: Genera números de 0 a 999 y usa el residuo entre dos para agregarlos a las listas de pares e impares.

Salidas: Lista original y las dos listas clasificadas.

Funcionamiento y evidencia: Introduce 20 y pulsa Generar y separar. La captura muestra las tres listas; ningún número se pierde. Cero, decimales y letras se rechazan como cantidad.



L3. Aprobados y reprobados

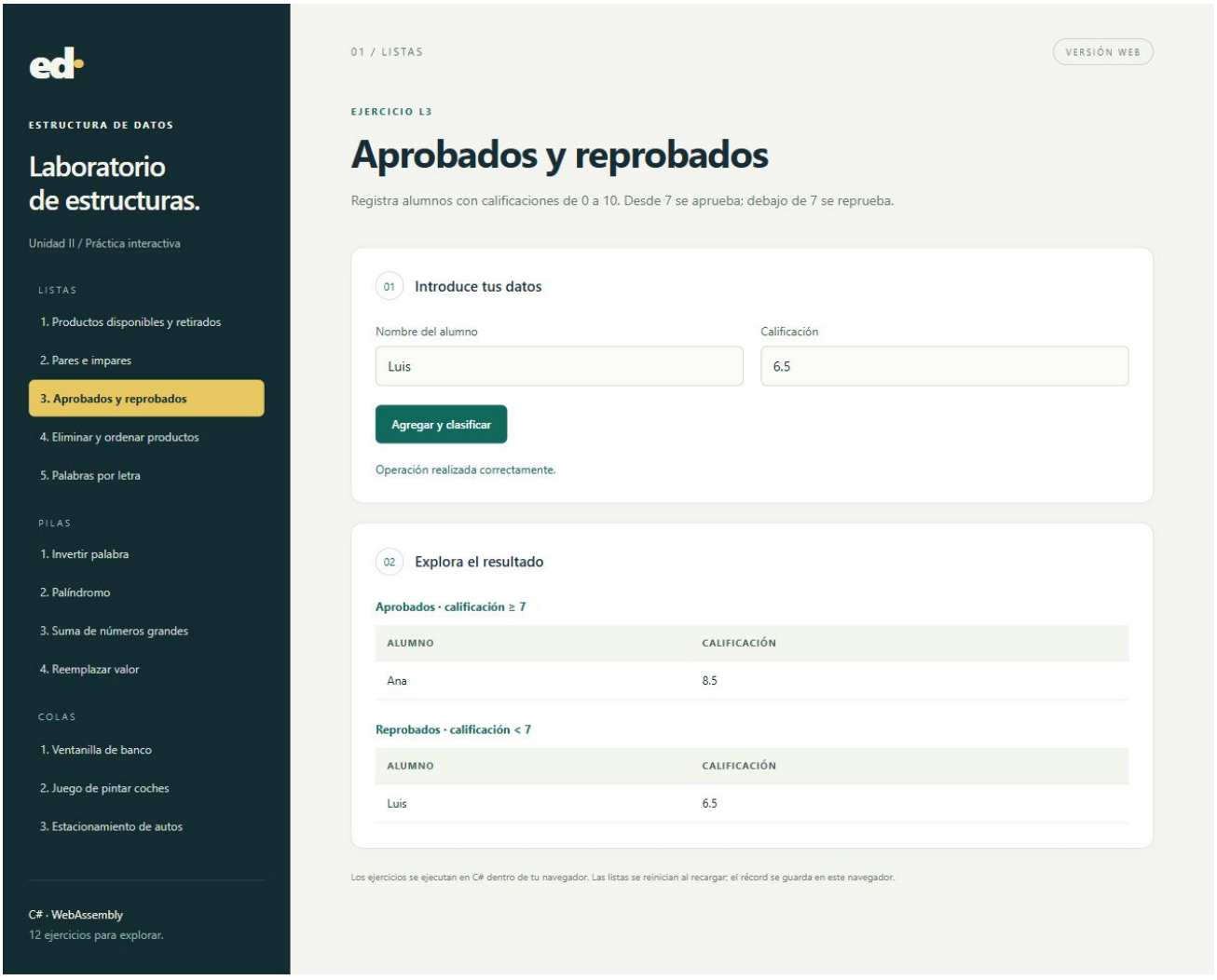
Problema: Clasificar alumnos según su calificación.

Entradas: Nombre y calificación decimal de 0 a 10.

Proceso: Agrega al alumno y extrae de la lista inicial los que tengan calificación menor que siete, trasladándolos a reprobados.

Salidas: Listas de aprobados y reprobados con sus notas.

Funcionamiento y evidencia: Registra Ana con 8.5 y Luis con 6.5. La captura muestra a cada alumno en su lista. Una nota igual a 7 aprueba; un nombre vacío o nota fuera de rango genera error.



L4. Eliminar y ordenar productos

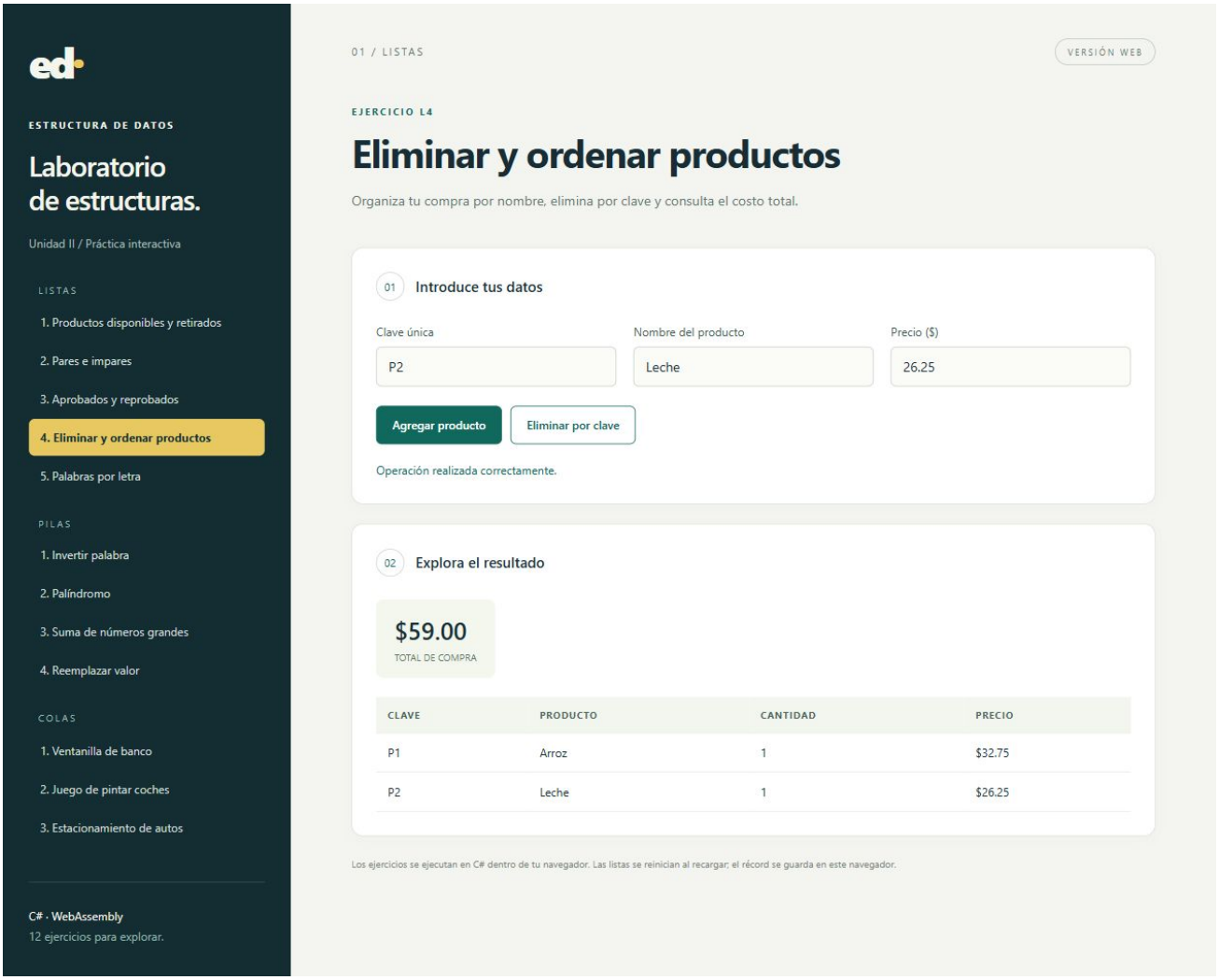
Problema: Mantener una compra con eliminación por clave y total.

Entradas: Clave única, nombre y precio decimal no negativo.

Proceso: Agrega a una lista, elimina por clave, ordena por nombre con comparación en español y suma precios usando decimal.

Salidas: Tabla ordenada y costo total.

Funcionamiento y evidencia: Agrega Arroz de 32.75 y Leche de 26.25. La imagen muestra total 59.00. Para eliminar, escribe la clave y pulsa Eliminar por clave. Se rechazan claves duplicadas y precios negativos.



L5. Palabras por letra

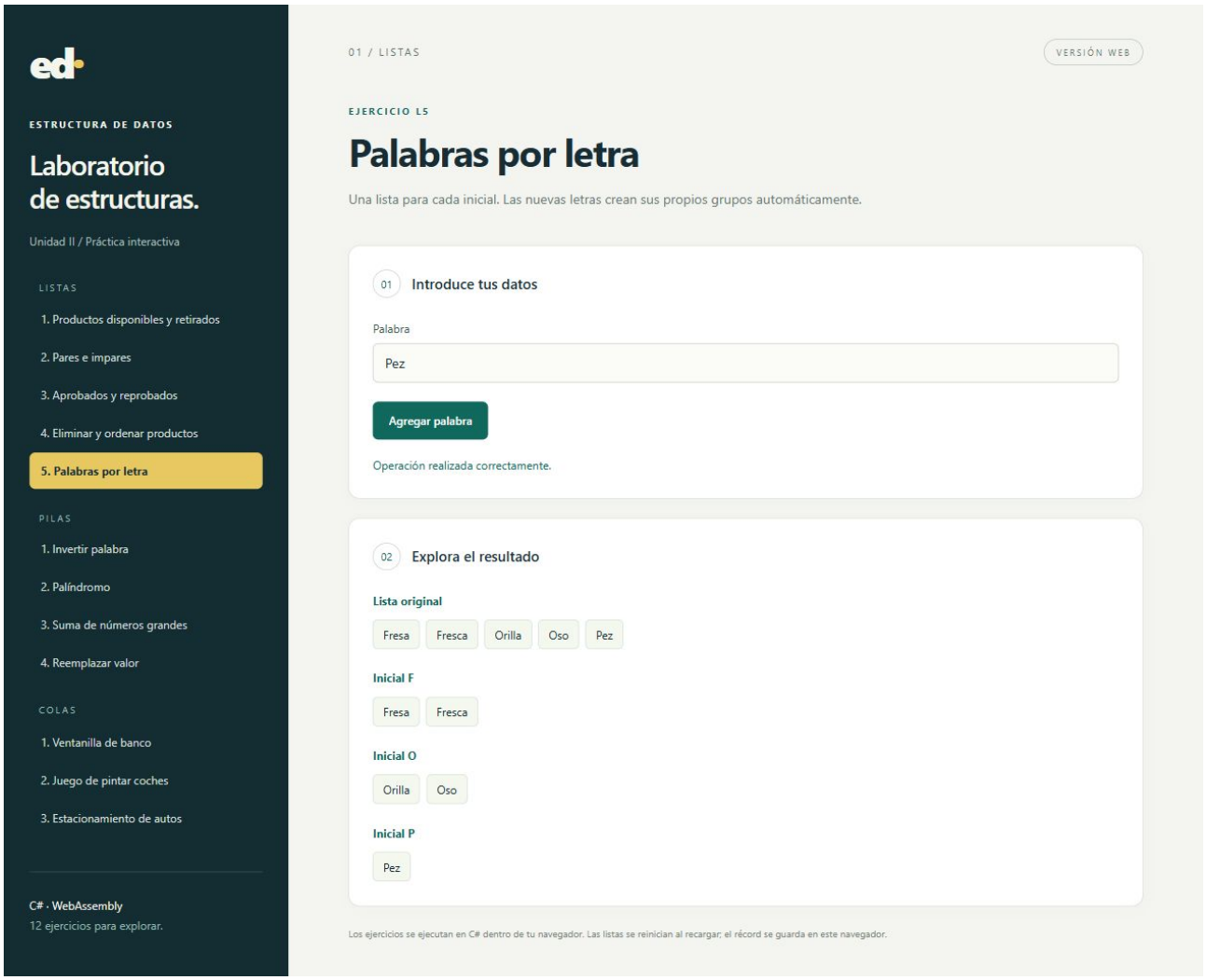
Problema: Crear listas dinámicas según la primera letra de cada palabra.

Entradas: Una palabra formada por letras en cada captura.

Proceso: Conserva la lista original y agrupa por inicial en un diccionario ordenado cuyos valores son List<string>.

Salidas: Lista original y una lista por inicial.

Funcionamiento y evidencia: Agrega Fresa, Fresca, Oso, Orilla y Pez. La captura muestra grupos F, O y P. Nuevas iniciales crean listas automáticamente; se respetan letras acentuadas.



P1. Invertir palabra

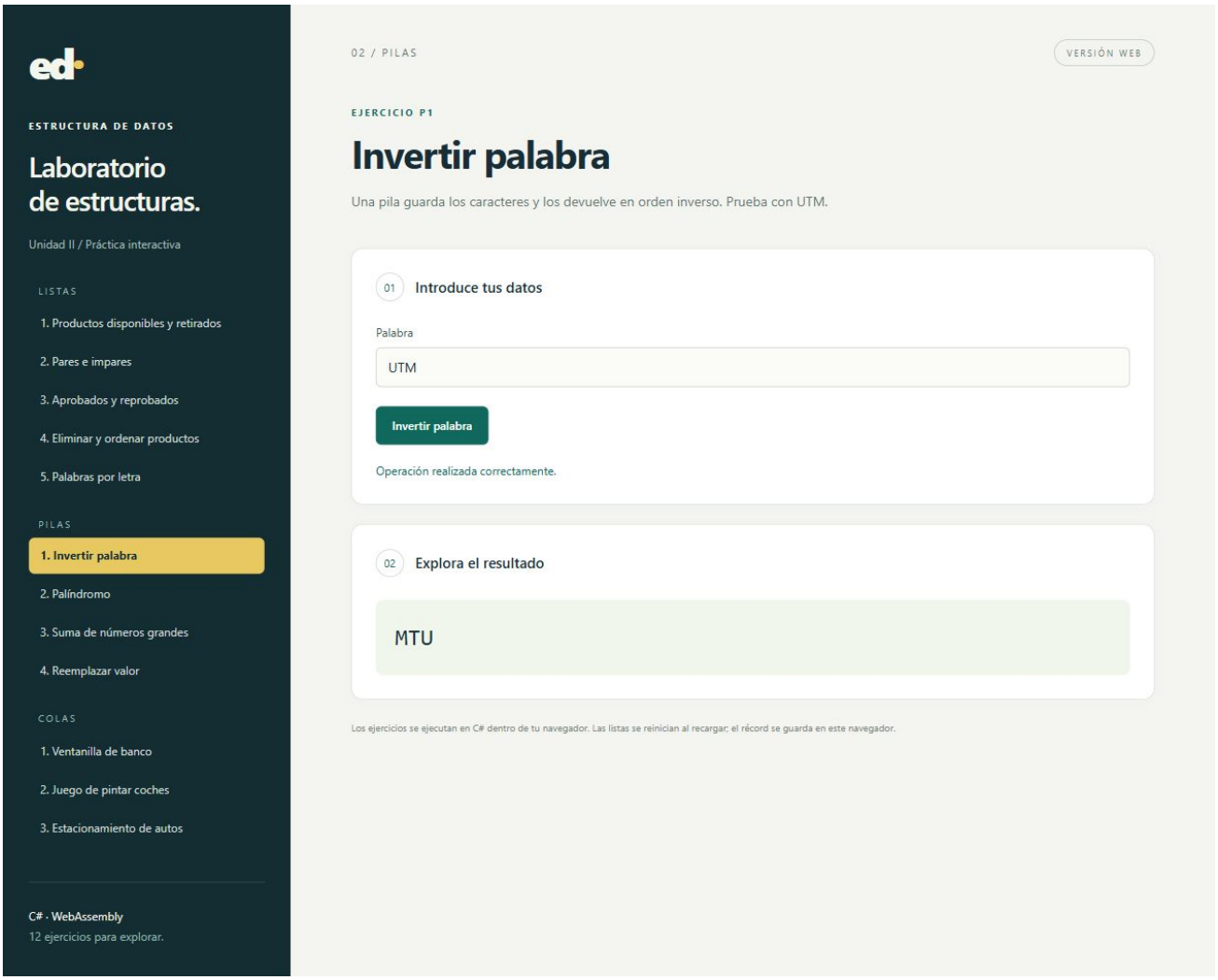
Problema: Mostrar una palabra en orden inverso mediante una pila.

Entradas: Palabra no vacía.

Proceso: Apila cada Rune del texto y desapila hasta vaciar la estructura.

Salidas: Texto invertido, preservando los caracteres Unicode completos.

Funcionamiento y evidencia: Escribe UTM y pulsa Invertir. La captura muestra MTU. El texto vacío genera un mensaje de datos faltantes.



P2. Palíndromo

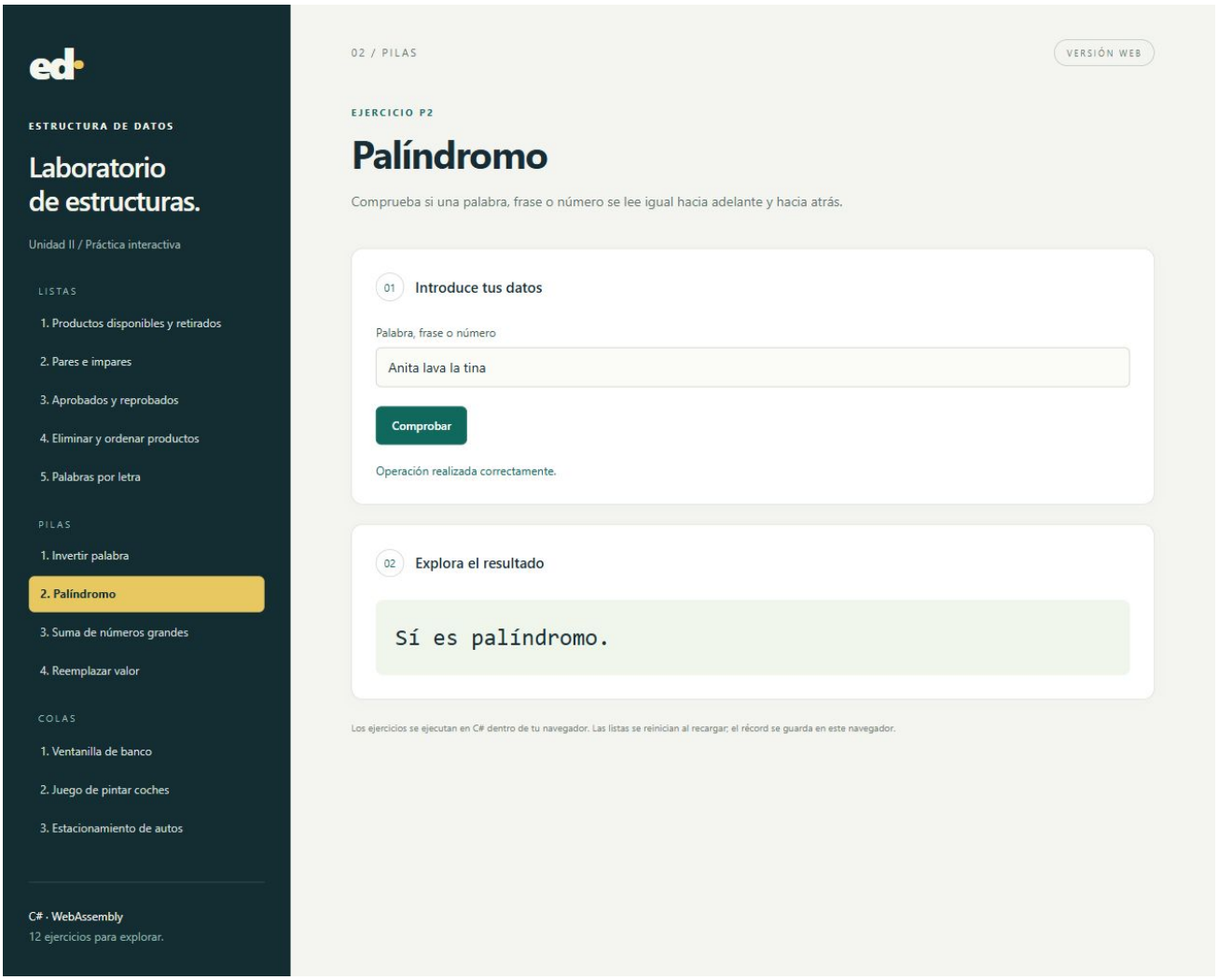
Problema: Determinar si un texto se lee igual en ambos sentidos.

Entradas: Palabra, frase o número.

Proceso: Normaliza mayúsculas y elimina espacios y signos; compara con la inversión realizada mediante pila.

Salidas: Mensaje afirmativo o negativo.

Funcionamiento y evidencia: Introduce Anita lava la tina y pulsa Comprobar. La captura muestra que es palíndromo. Se conservan acentos y se rechazan entradas compuestas únicamente por signos.



P3. Suma de números grandes

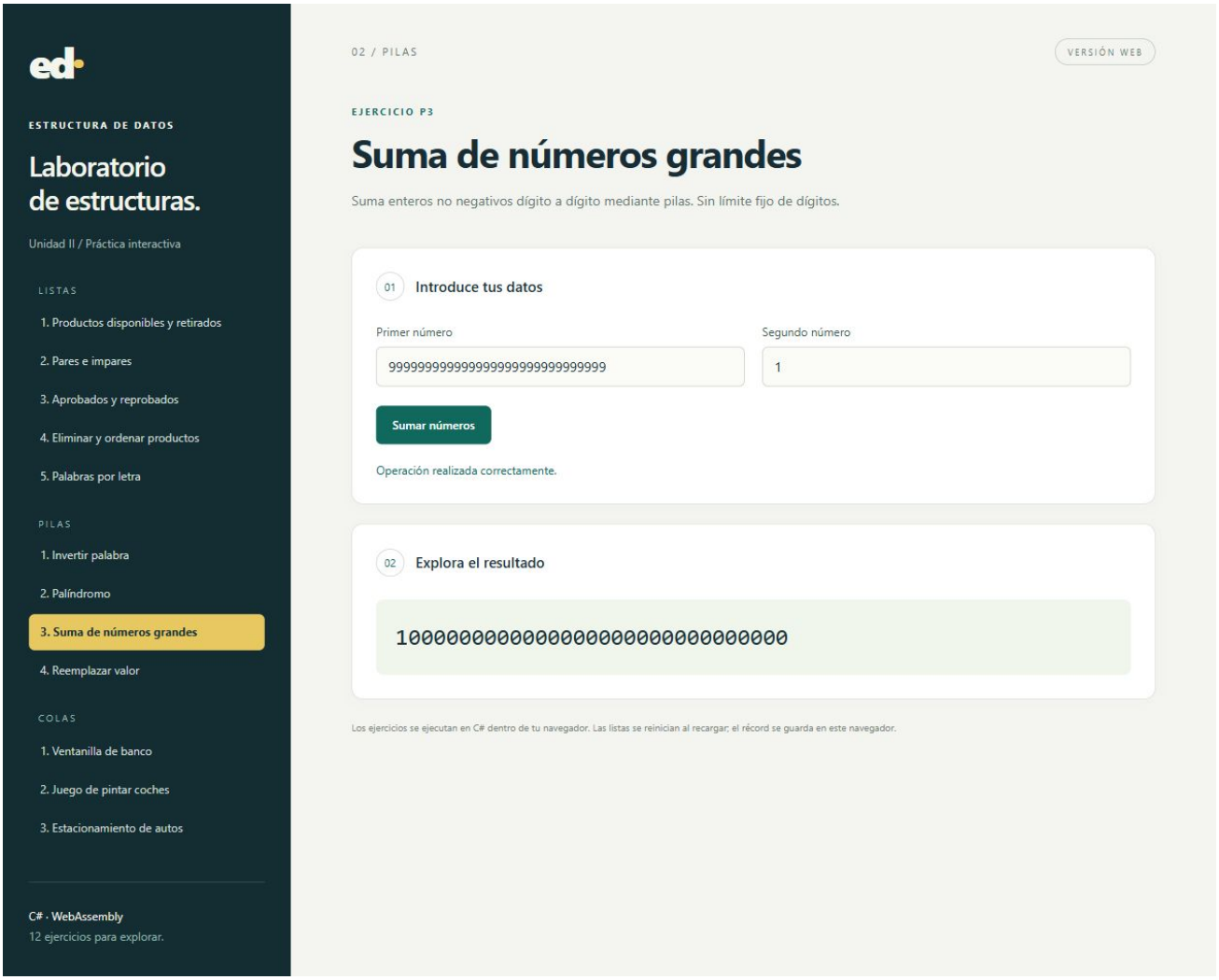
Problema: Sumar enteros que exceden los tipos numéricos ordinarios.

Entradas: Dos cadenas de dígitos, no negativas y sin separadores.

Proceso: Apila dígitos de ambos operandos, desapila de derecha a izquierda, suma con acarreo y construye el resultado con una tercera pila.

Salidas: Suma completa sin límite fijo de dígitos.

Funcionamiento y evidencia: Introduce treinta nueves y 1, y pulsa Sumar números. La imagen muestra un uno seguido de treinta ceros. Letras y signos se rechazan. BigInteger sólo se usa para comprobar las pruebas.



P4. Reemplazar valor

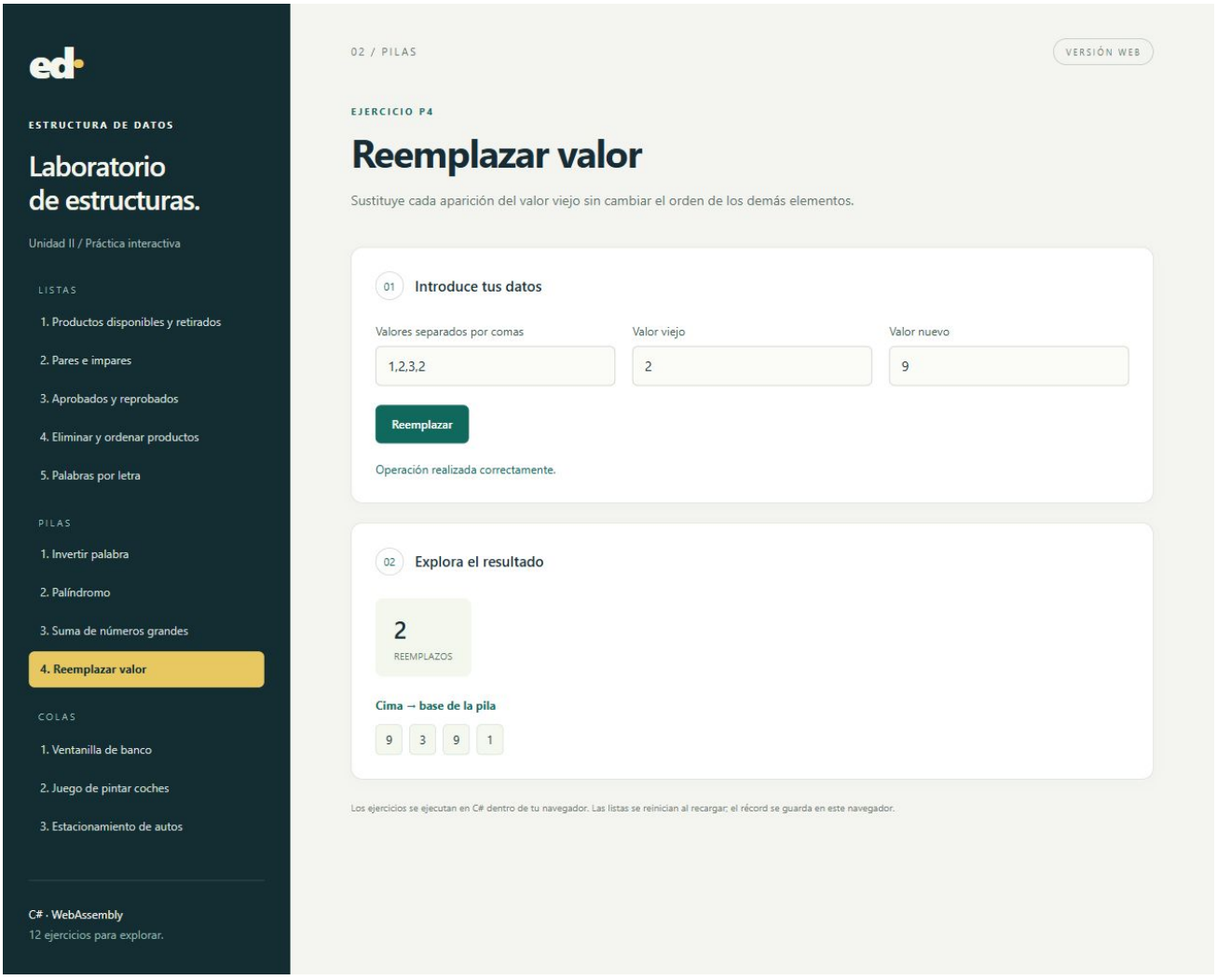
Problema: Sustituir un entero por otro en una pila sin alterar su orden.

Entradas: Enteros separados por comas, valor viejo y valor nuevo.

Proceso: Desapila hacia una pila auxiliar sustituyendo coincidencias; después restaura la pila original.

Salidas: Pila desde la cima y cantidad de reemplazos.

Funcionamiento y evidencia: Con valores 1,2,3,2, viejo 2 y nuevo 9, pulsa Reemplazar. La captura muestra 9,3,9,1 desde la cima y dos reemplazos. Se interpreta el error de redacción del enunciado como reemplazar viejo por nuevo.



C1. Ventanilla de banco

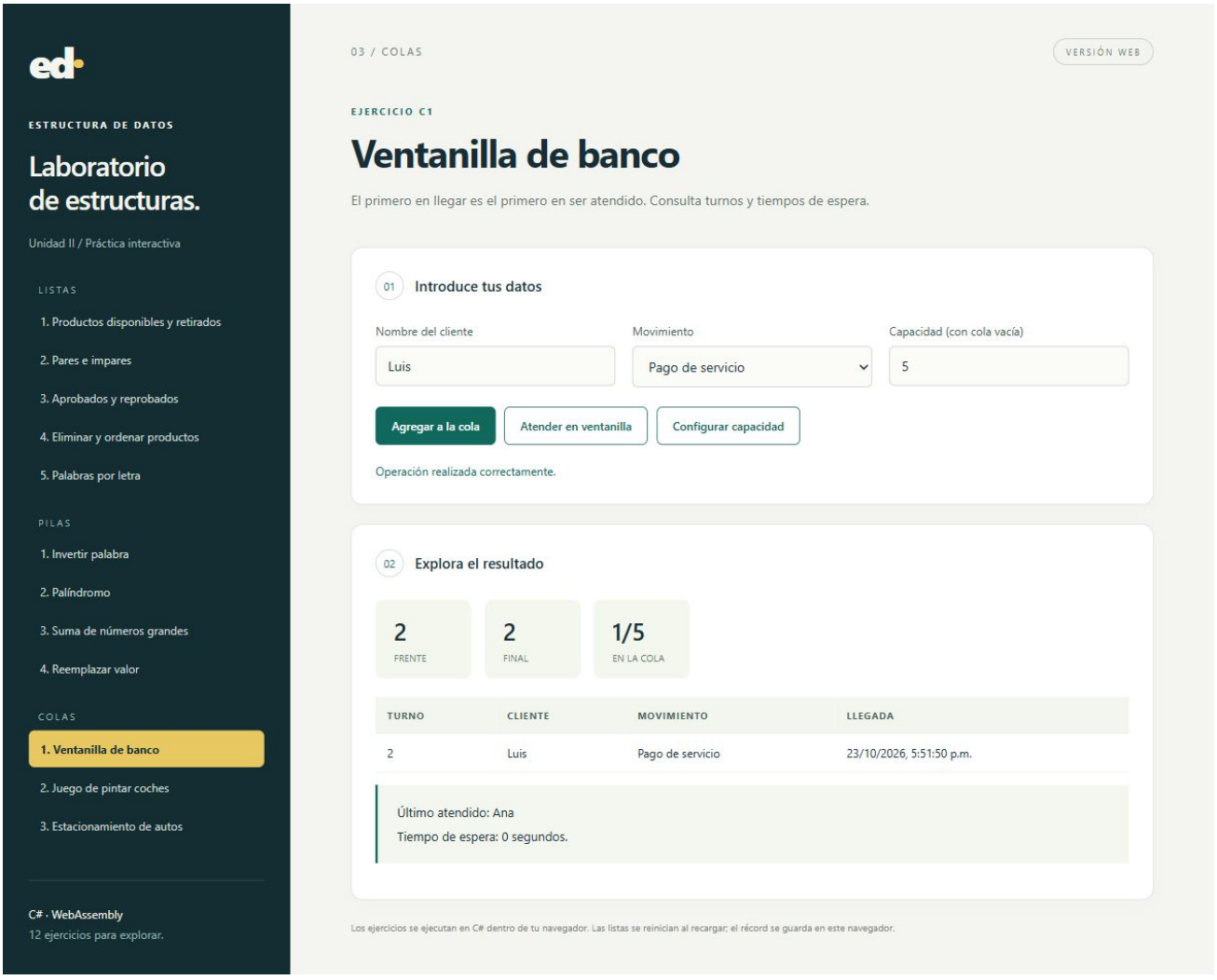
Problema: Administrar clientes mediante cola FIFO con capacidad limitada.

Entradas: Nombre, movimiento del ComboBox y capacidad configurable con cola vacía.

Proceso: Asigna turno y hora DateTime.Now; InsertaCola valida ColaLlena. Atender usa EliminaCola tras validar ColaVacia y calcula la espera.

Salidas: DataGridView actualizado, frente y final, confirmación de llegada y tiempo de espera.

Funcionamiento y evidencia: Registra Ana y Luis y atiende a Ana. La captura deja a Luis en la cola y muestra el tiempo real de espera. La capacidad inicial es cinco; se validan sobreflujo y atención con cola vacía.



C2. Juego de pintar coches

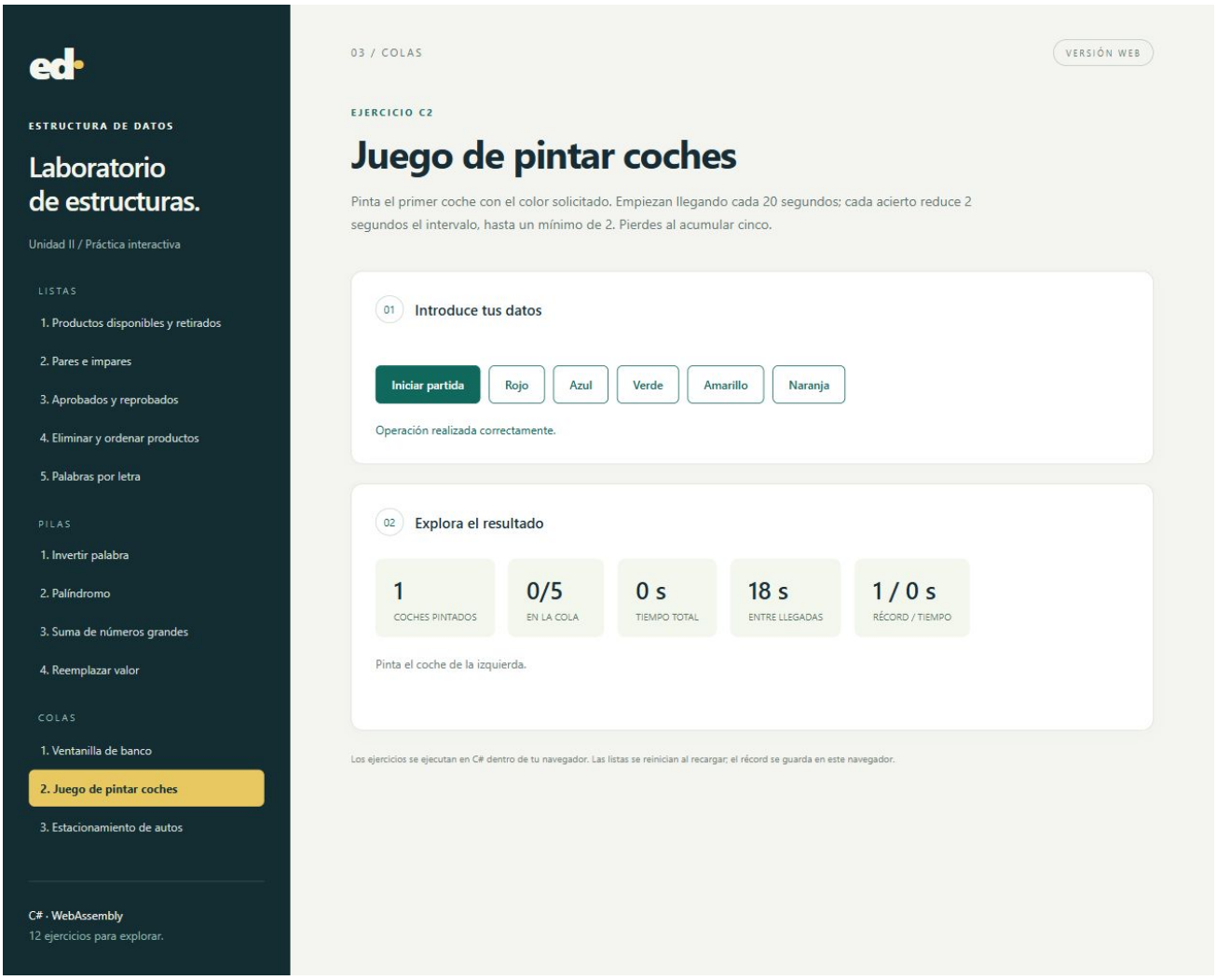
Problema: Atender coches en orden y pintarlos del color solicitado antes de que se acumulen cinco.

Entradas: Botón Iniciar partida y selección de color de la paleta de botones.

Proceso: Un reloj monótono controla llegadas y tiempo de partida. El primer coche aparece inmediatamente y el intervalo comienza en 20 segundos. Cada coche correctamente pintado reduce 2 segundos el intervalo y adelanta la siguiente llegada si corresponde, hasta un mínimo de 2 segundos. Sólo puede retirarse el frente.

Salidas: Dibujos de coches con etiqueta de color, puntuación, tiempo y récord.

Funcionamiento y evidencia: Pulsa Iniciar partida y luego el color solicitado por el coche de la izquierda. La captura muestra un acierto y el intervalo reducido de 20 a 18 segundos. Los siguientes aciertos lo reducen a 16, 14 y así hasta 2 segundos. La partida termina al acumular cinco coches. El récord y su tiempo se guardan en este navegador.



C3. Estacionamiento de autos

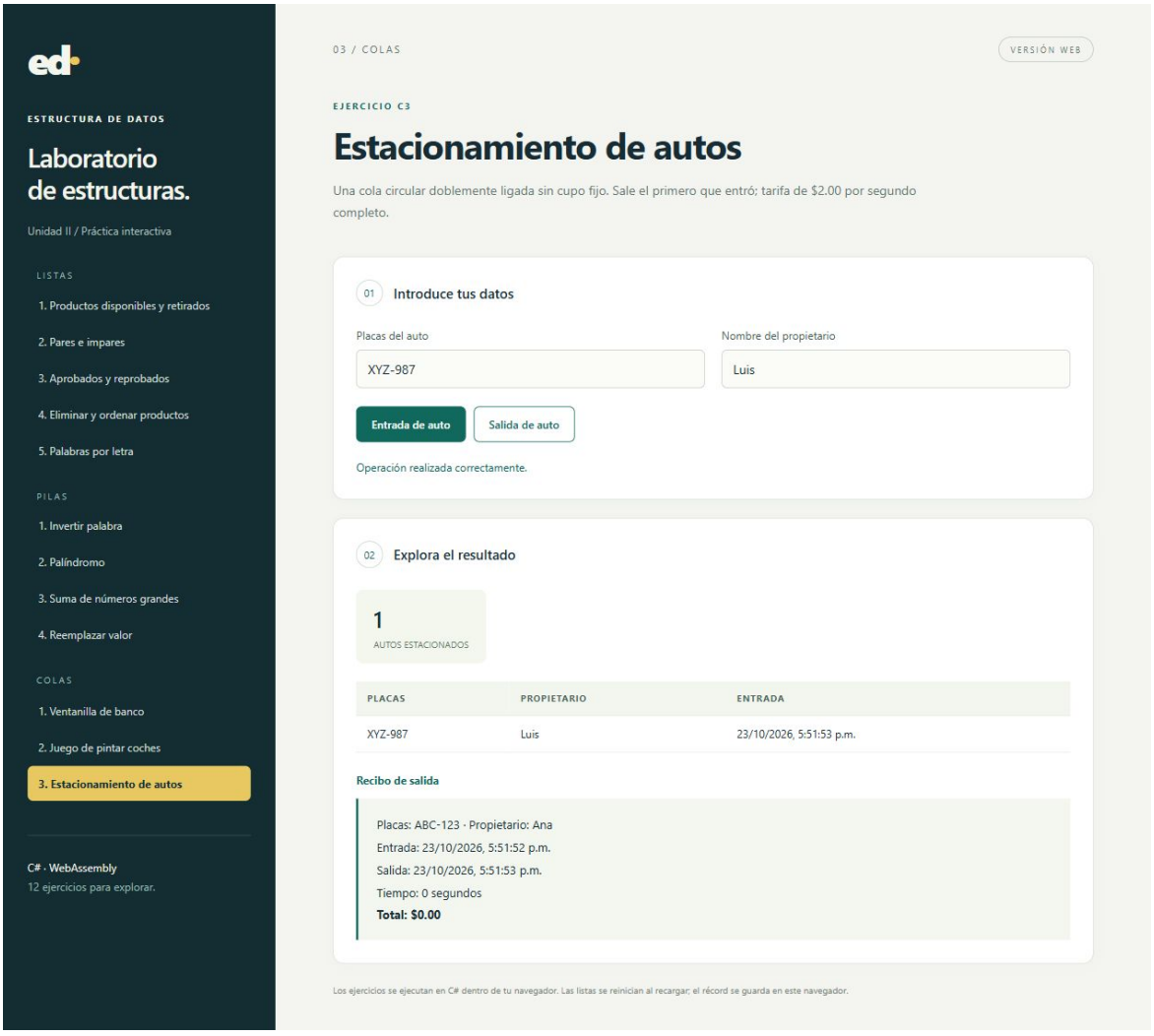
Problema: Controlar autos en un estacionamiento sin cupo fijo y cobrar dos pesos por segundo completo.

Entradas: Placas y propietario; hora de entrada y salida tomadas del sistema.

Proceso: Inserta cada auto como nodo de una cola circular doblemente ligada. Salir retira el frente, repara enlaces y calcula segundos transcurridos por 2.00.

Salidas: Tabla de autos y recibo con placas, propietario, horas, tiempo y costo.

Funcionamiento y evidencia: Registra ABC-123 y XYZ-987; pulsa Salida de auto. La captura muestra el recibo del primero y el segundo todavía en cola. Se rechazan placas repetidas y salida con estacionamiento vacío.



Pruebas y decisiones

Se verificaron los doce ejercicios por sus controles en la versión C# WebAssembly y el cambio de dificultad después del primer acierto. La variante ASP.NET además tiene pruebas HTTP de validación y aislamiento de sesiones.

pruebas-publica.txt

OK | L1: controles y resultado C# WebAssembly

OK | L2: controles y resultado C# WebAssembly

OK | L3: controles y resultado C# WebAssembly

OK | L4: controles y resultado C# WebAssembly

OK | L5: controles y resultado C# WebAssembly

OK | P1: controles y resultado C# WebAssembly

OK | P2: controles y resultado C# WebAssembly

OK | P3: controles y resultado C# WebAssembly

OK | P4: controles y resultado C# WebAssembly

OK | C1: controles y resultado C# WebAssembly

OK | C2: controles y resultado C# WebAssembly

OK | Dificultad: el primer acierto reduce 20 a 18 segundos

OK | C3: controles y resultado C# WebAssembly

TOTAL: 12 ejercicios web verificados en navegador; dificultad progresiva comprobada.

pruebas-web.txt

OK | Página web disponible

OK | L1: escanear y retirar

OK | L2: clasificación completa

OK | L3: límite de aprobación

OK | L4: orden y total decimal

OK | L4: eliminar por clave

OK | L5: grupos por letra

OK | P1: invertir palabra

OK | P2: palíndromo

OK | P3: acarreo de mil dígitos

OK | P4: reemplazar conservando el orden

OK | C1: rechazar sobreflujo

OK | C1: atención FIFO y cola vacía

OK | C2: partida y coche inicial

OK | C2: pintar el primer coche

OK | C2: el primer acierto reduce el intervalo a 18 segundos

OK | C3: salida FIFO y recibo

OK | Validación: nota fuera de rango

OK | Validación: suma inválida

OK | Validación: placas repetidas

OK | Sesiones: datos separados por navegador

OK | Solicitudes: cabecera requerida

TOTAL: 22 verificaciones web correctas. Sin fallos.

En la versión pública los datos de los ejercicios permanecen al navegar y se reinician al recargar. El récord del juego se conserva en el almacenamiento local. La variante ASP.NET usa sesiones del servidor, con expiración por inactividad o reinicio. La tarifa de estacionamiento es de $2.00 por segundo completo; la salida es FIFO.

Se resuelven los errores de redacción del enunciado como: mostrar aprobados y reprobados, y sustituir el valor viejo por el nuevo manteniendo el orden de la pila. El cambio de dificultad por cada coche pintado fue solicitado expresamente por el equipo.

GUÍA DEL CÓDIGO / VERSIÓN WEB

Cómo consultar y explicar este trabajo

El desarrollo y las evidencias de los doce ejercicios se conservan en las páginas 2 a 15. Esta guía amplía la explicación del código que realmente ejecuta la práctica: estructuras utilizadas, métodos, datos que reciben, decisiones, resultados y ejemplos que pueden reproducirse durante la exposición.

Parte del documento	Páginas
Portada enviada por el equipo	1
Desarrollo y evidencias originales	2 a 13
Pruebas y decisiones del desarrollo	14 y 15
Arquitectura, interfaz y conceptos comunes	16 a 20
Explicación de los doce ejercicios	21 a 44
Tiempo del juego, servidor, pruebas y exposición	45 a 49
Mapa de archivos y líneas de inicio	50

Cómo leer cada ejercicio

La primera página identifica la estructura de datos, explica los métodos y presenta un fragmento real del programa. La segunda recorre una ejecución con un ejemplo, muestra el estado de los datos y explica las validaciones. Al final incluye una explicación breve que puede adaptarse a la exposición oral.

Datos del equipo

Santiago Nicolás De la Mora Núñez; Jesus Joel Canche Chac; Roger Alejandro Aguilar Nuñez; Víctor Manuel Almendarez García

Grupo: 4.º A. Docente: Ing. Mirian Magaly Canché Caamal. Fecha de entrega: 23 de octubre de 2026.

Qué representa un fragmento de código

Los fragmentos se contrastan automáticamente con CSharp/Modelos.cs. Sólo se ajustan sus saltos de línea para que se lean en la página. No son pseudocódigo ni una implementación diferente. Las tablas de recorrido son ejemplos didácticos; los valores aleatorios de las capturas pueden variar.

Arquitectura: qué hace cada archivo

Archivo	Responsabilidad
CSharp/Modelos.cs	Contiene Producto, Alumno, Almacen, Grupo, Compra, Algoritmos, Cola<T>, Banco, Juego y los nodos del estacionamiento. Aquí se resuelven los ejercicios.
Web/Estado.cs	Conserva las instancias de los modelos, interpreta módulo y acción, llama al método adecuado y prepara Vista(), que devuelve el estado actual.
WebPublica/Program.cs	Arranca Blazor WebAssembly y expone Consultar, Ejecutar y RestaurarRecord para que JavaScript llame al código C#.
WebPublica/wwwroot/motor.js	Espera a Blazor.start, usa DotNet.invokeMethodAsync y guarda o recupera el récord del navegador.
Web/wwwroot/app.js	Construye formularios, captura eventos, pide operaciones y dibuja tablas, listas, coches y mensajes. Se reutiliza en la publicación.
Web/Program.cs	Aloja la variante ASP.NET Core. Expone las rutas HTTP y conserva un Estado por sesión de navegador.

Recorrido de la versión pública

Formulario HTML > app.js > motor.js > MotorWeb.Ejecutar > Estado.Ejecutar > modelo C# > Estado.Vista > JSON > render de app.js > pantalla.

GitHub Pages entrega archivos estáticos. El motor .NET se descarga y se ejecuta dentro del navegador gracias a WebAssembly. No hay un servidor C# de GitHub Pages atendiendo cada clic: los algoritmos C# se ejecutan en el dispositivo de quien abre la página.

Por qué compartir los modelos

Los proyectos incluyen Modelos.cs y Estado.cs mediante referencias Compile. Esto permite reutilizar la misma lógica sin copiar cada algoritmo a otra implementación en JavaScript. Cambiar una regla en el modelo afecta a los proyectos que se vuelvan a compilar con esa fuente.

Del clic al resultado: funciones de la interfaz

choose(m)

Selecciona el ejercicio, actualiza el título, la descripción, los campos y los botones. No crea un modelo C# nuevo; por eso cambiar de pestaña conserva los datos de los ejercicios.

act(action, label)

Lee FormData del formulario, identifica el módulo y construye un objeto con modulo, accion y valores. En el juego añade el color del botón. Usa busy y deshabilita botones mientras procesa una operación para evitar clics simultáneos.

sendAction(solicitud)

Si existe practicaMotorReady, espera el motor WebAssembly y ejecuta su acción. En la variante ASP.NET envía un POST a /api/accion con JSON y la cabecera X-Practica.

MotorWeb.Ejecutar(json)

Convierte el JSON a una Solicitud de C#, valida tamaño y valores, llama a Estado.Ejecutar y devuelve mensaje y estado serializados. Los errores de validación se convierten en un objeto error.

Estado.Ejecutar(s)

El switch combina módulo y acción, por ejemplo L1/retirar o P3/sumar. Obtiene los valores por nombre, los valida y llama únicamente al modelo que corresponde al ejercicio.

Vista() y render()

Vista prepara datos de salida y copias de las colecciones. render toma ese estado y dibuja el resultado del ejercicio seleccionado. La pantalla presenta lo que decidió el modelo, en vez de calcular por su cuenta la suma o la cola.

refresh()

Consulta el estado actual. Un setInterval pide esta actualización cada segundo cuando el juego está seleccionado. No significa que el algoritmo del juego avance exactamente una vez por segundo; el modelo usa el tiempo real transcurrido.

JSON es el formato de intercambio: permite representar datos como objetos, listas, números y texto. Serializar convierte los objetos C# a JSON; deserializar reconstruye la Solicitud recibida.

Validaciones y tipos de datos

Función o tipo	Qué hace y por qué importa
Validar.Texto(valor)	Rechaza null, cadena vacía y sólo espacios; después aplica Trim para quitar espacios exteriores. No elimina los espacios interiores de un nombre.
Validar.Entero(valor, min, max)	Usa int.TryParse. Distingue una conversión fallida de un entero que sí pudo leerse pero quedó fuera de rango. Se usa para cantidades, capacidad y elementos de pila.
Validar.Decimal(valor, min, max)	Sustituye coma por punto y analiza con cultura invariante. Admite signo inicial y separador decimal, sin interpretar separadores de miles. Comprueba el rango antes de devolver el valor.
decimal	Se usa en dinero y calificaciones. Mantiene precisión decimal para cantidades como 15.50 y 32.75. La interfaz muestra dinero con dos decimales.
string	Conserva texto y los operandos grandes. SumarGrandes no convierte toda la cadena a int: sólo convierte cada dígito individual.
int y long	int guarda contadores y valores enteros. long permite contar milisegundos y segundos con un rango mayor en la temporización.
DateTime y TickCount64	DateTime representa una hora del sistema para banco y estacionamiento. TickCount64 es un reloj de tiempo transcurrido, apropiado para programar llegadas del juego.

Cómo se muestra un error

El modelo lanza ArgumentException con un mensaje comprensible cuando no puede cumplir una regla. El motor o la API capturan esa excepción y la interfaz la muestra en el estado del formulario. Un OverflowException produce un mensaje específico de rango numérico.

Una validación correcta ocurre antes de modificar la colección cuando sea posible. Por ejemplo, no se agrega un producto con clave duplicada y no se elimina el frente del juego si el color solicitado es incorrecto.

Listas, pilas y colas: conceptos que usamos

Estructura	Operaciones del código	Regla
List<T>	Add, Remove, Find, Where, OrderBy	Guarda una secuencia. Permite buscar o eliminar por una condición; no impone FIFO ni LIFO.
Stack<T>	Push, Pop, Count y construcción desde una secuencia	LIFO: el último que entra sale primero. Enumerar una Stack muestra primero su cima.
Queue<T>	Enqueue, Dequeue, Peek	FIFO: se agrega al final y se retira del frente. La clase Cola<T> añade comprobaciones de cupo.
SortedDictionary<K,V>	ContainsKey, acceso por clave y Add a su lista	Relaciona cada inicial con una lista. Ordena las claves, no necesariamente los elementos de cada lista.
Nodo circular doble	Anterior, Siguiente y frente	Conecta cada nodo en dos sentidos y cierra el circuito. Requiere mantener los enlaces al insertar y retirar.

Qué significan las expresiones del código

Count es la cantidad de elementos. Una comparación como x == viejo comprueba igualdad. El operador % obtiene el residuo; / entre enteros descarta la parte fraccionaria. La expresión condición ? a : b elige uno de dos valores. ?? usa la alternativa cuando el resultado de la izquierda es null.

Una expresión lambda, como p => p.Clave == clave, representa una regla que recibe un producto y devuelve una condición. Where selecciona elementos; Any pregunta si existe alguno; Sum suma valores. ToList o ToArray crean una colección materializada para usarla después.

Orientación a objetos en esta práctica

Un record como Producto o Cliente reúne datos de un elemento. Una clase como Banco mantiene su propia colección y las operaciones permitidas sobre ella. El controlador coordina las clases, y la interfaz se encarga de capturar y presentar. Esta división ayuda a explicar y probar los algoritmos por separado.

L1. Productos disponibles y retirados: métodos

Producto y Almacen

Dos List<Producto>: Disponibles representa el almacén actual y Retirados conserva el historial. Cada Producto es un record con Clave, Nombre, Cantidad y Precio.

Escanear()

Incrementa secuencia con ++ para generar una clave distinta, como P1. Construye el nombre producto1, obtiene una cantidad aleatoria de 1 a 20 y un precio de $1.00 a $500.00. Random.Shared.Next excluye el límite superior: por eso se usan 21 y 50001. Divide entre 100m para obtener un decimal monetario y agrega el objeto con Add.

Retirar(clave)

Limpia la clave con Validar.Texto. Find recorre disponibles y compara sin distinguir mayúsculas. Si no encuentra el producto, ?? throw genera un error. Si existe, Remove lo quita de disponibles y Add incorpora el mismo objeto al historial; no se pierde su cantidad ni su precio.

Presentación

Estado.Ejecutar conecta L1/escanear y L1/retirar con esos métodos. Vista devuelve copias de ambas listas y render dibuja dos tablas. Los contadores muestran el número de objetos Producto, no la suma de unidades del campo Cantidad.

Fragmento real que conviene explicar

```
string k = Validar.Texto(clave);
var p = Disponibles.Find(p => p.Clave.Equals(k,
    StringComparison.OrdinalIgnoreCase))
    ?? throw new ArgumentException(
        "Clave no encontrada entre disponibles.");
Disponibles.Remove(p);
Retirados.Add(p);
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 25, 32, 39.

L1. Recorrido y ejemplo para exponer

1. El botón Escanear envía el módulo L1 y la acción escanear. La primera llamada crea P1 y la segunda P2; sus cantidades y precios son aleatorios.

2. El usuario escribe P1 y pulsa Retirar por clave. El controlador pasa el texto a Almacen.Retirar.

3. Find localiza P1. Se elimina únicamente de Disponibles y se añade a Retirados. P2 permanece disponible.

4. Vista prepara el estado actualizado; la página vuelve a pintar las tablas y cambia los contadores a uno disponible y uno retirado.

Estado de los datos durante la ejecución

Operación	Disponibles	Retirados
Inicio	vacía	vacía
Escanear dos veces	P1, P2	vacía
Retirar P1	P2	P1

Validaciones y límites

Una clave vacía se rechaza antes de buscar. Una clave inexistente o ya retirada muestra el mensaje correspondiente y no modifica ninguna lista.

Cómo explicarlo con nuestras palabras

Usamos dos listas para distinguir la existencia actual del historial. Retirar no destruye el registro: lo mueve conservando sus datos. Podemos demostrarlo escaneando dos productos y retirando sólo P1.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/L1.png.

L2. Pares e impares: métodos

Algoritmos.Clasificar

Tres List<int>: Original conserva cada número generado; Pares e Impares reciben copias de esos valores según su paridad. Se permiten repetidos.

Clasificar(cantidad)

Primero exige una cantidad entre 1 y 500. El ciclo for se repite exactamente cantidad veces. En cada vuelta Next(1000) genera un entero entre 0 y 999, lo añade a Original y lo coloca en una sola lista de clasificación.

n % 2 == 0

El operador % calcula el residuo. Un residuo cero indica que n es divisible entre dos y, por tanto, es par. La expresión condicional elige Pares o Impares; Add guarda el número en la lista elegida.

Resultado devuelto

El método devuelve una tupla con las tres listas. Estado convierte sus miembros en un objeto que se puede serializar. La interfaz muestra cada lista por separado sin perder el orden de generación dentro de cada grupo.

Fragmento real que conviene explicar

```
List<int> original = [], pares = [], impares = [];
for (int i = 0; i < cantidad; i++)
{
    int n = Random.Shared.Next(1000);
    original.Add(n);
    (n % 2 == 0 ? pares : impares).Add(n);
}
return (original, pares, impares);
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 80.

L2. Recorrido y ejemplo para exponer

1. Se lee la cantidad del formulario y Validar.Entero la convierte a un entero dentro del rango.

2. Por cada vuelta del for se genera un número. Original siempre recibe ese número, aunque ya exista otro igual.

3. La condición del residuo decide el destino. Como las opciones son excluyentes, cada número aparece en una sola de las dos listas de clasificación.

4. Al terminar, la suma de las cantidades de pares e impares coincide con la cantidad original. El ejemplo de abajo es ilustrativo: una ejecución real genera otros números.

Estado de los datos durante la ejecución

Número ilustrativo	Residuo entre 2	Destino
8	0	Pares
3	1	Impares
0	0	Pares
3	1	Impares; se conserva el repetido

Validaciones y límites

Cero, cantidades mayores que 500, letras o cantidades decimales se rechazan antes de generar la lista. Generar otra vez sustituye el resultado anterior de este ejercicio.

Cómo explicarlo con nuestras palabras

Recorremos los números una sola vez y usamos el residuo para separarlos. Comprobamos que ningún elemento se pierde: pares más impares debe ser igual al total generado.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/L2.png.

L3. Aprobados y reprobados: métodos

Alumno y Grupo

Alumno conserva Nombre y Calificacion como decimal. Grupo tiene dos List<Alumno>: Aprobados y Reprobados.

Agregar(nombre, nota)

Comprueba que la nota esté entre cero y diez y valida el nombre. Agrega inicialmente el alumno a Aprobados. Después busca dentro de esa lista a quienes tengan nota menor que siete y los traslada a Reprobados.

Where(...).ToList()

Where selecciona únicamente las notas menores que siete. ToList crea una lista temporal de esos alumnos. Es importante porque se va a eliminar de la lista original: recorrer directamente una colección mientras se modifica puede invalidar su enumerador.

Remove y Add

Por cada alumno seleccionado se ejecutan ambas operaciones. Al concluir, nadie con nota menor que siete permanece en Aprobados. Una nota igual a siete sí aprueba; no se redondea 6.99 para convertirla en siete.

Fragmento real que conviene explicar

```
Aprobados.Add(new Alumno(Validar.Texto(nombre), nota));
foreach (var a in Aprobados.Where(
    a => a.Calificacion < 7).ToList())
{
    Aprobados.Remove(a);
    Reprobados.Add(a);
}
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 26, 48.

L3. Recorrido y ejemplo para exponer

1. La pantalla envía el nombre y la nota. Validar.Decimal admite punto o coma decimal, por ejemplo 8.5 y 8,5.

2. Se construye el record Alumno y se agrega a la lista de aprobados, que funciona como lista inicial de captura.

3. Si la nota es menor que siete, el foreach la encuentra en la selección temporal y mueve el alumno a la otra lista.

4. Vista devuelve las dos listas y render dibuja dos tablas. El mismo nombre puede capturarse más de una vez: esta versión no establece una matrícula única.

Estado de los datos durante la ejecución

Alumno	Nota	Lista final
Ana	7.00	Aprobados
Luis	6.99	Reprobados
Eva	10.00	Aprobados
Pedro	0.00	Reprobados

Validaciones y límites

Un nombre vacío o una calificación fuera del intervalo 0 a 10 produce un mensaje sin agregar el alumno. El tipo decimal evita depender de aproximaciones binarias para comparar el límite de siete.

Cómo explicarlo con nuestras palabras

La regla es menor que siete reprueba. La parte importante es que primero obtenemos una copia de los alumnos que vamos a mover; así no modificamos la misma lista que estamos enumerando.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/L3.png.

L4. Eliminar y ordenar productos: métodos

Compra y Producto

Compra mantiene una List<Producto>. En este ejercicio cada registro representa un producto con cantidad uno; el precio de cada registro contribuye al total.

Agregar(clave, nombre, precio)

Valida la clave y comprueba con Any que no exista otra equivalente, sin distinguir mayúsculas. Rechaza precios negativos y nombres vacíos. Sólo después agrega un Producto a la lista. El controlador también impide que el total exceda el rango de decimal.

Eliminar(clave)

RemoveAll retira las coincidencias de la clave. Como las claves son únicas, normalmente elimina un solo producto. El valor devuelto es el número de eliminaciones; si es cero, se informa que la clave no existe.

Ordenados() y Total()

OrderBy construye una vista ordenada por nombre con comparación de español de México e ignorando mayúsculas. ToList la materializa sin ordenar físicamente la lista original. Sum suma todos los precios como decimal, y la pantalla los presenta con dos decimales.

Fragmento real que conviene explicar

```
string k = Validar.Texto(clave);
if (Productos.RemoveAll(p => p.Clave.Equals(k,
    StringComparison.OrdinalIgnoreCase)) == 0)
    throw new ArgumentException("Clave no encontrada.");
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 60, 75, 76.

L4. Recorrido y ejemplo para exponer

1. Se capturan clave, nombre y precio. Agregar comprueba sus reglas y guarda el producto.

2. Cada respuesta usa Ordenados para que la tabla se presente por nombre, aunque los productos se hayan capturado en otro orden.

3. Total vuelve a calcular la suma a partir de la lista actual. Por eso eliminar un producto también cambia el total sin llevar una suma paralela que pueda quedar desactualizada.

4. Para eliminar basta con escribir una clave y pulsar Eliminar por clave. Los otros campos del formulario no son necesarios en esa operación.

Estado de los datos durante la ejecución

Acción	Tabla por nombre	Total
Agregar P1 Zanahoria 15.50	Zanahoria	15.50
Agregar P2 Arroz 32.75	Arroz, Zanahoria	48.25
Eliminar P2	Zanahoria	15.50

Validaciones y límites

Se rechazan claves duplicadas, claves inexistentes al eliminar, precios negativos y textos faltantes. La validación de precio sólo corresponde a agregar; no debe impedir una eliminación por clave.

Cómo explicarlo con nuestras palabras

Separar almacenar de presentar nos permite conservar la lista y mostrar una copia ordenada. El total se calcula desde los productos vigentes, así que nunca incluye un producto que ya eliminamos.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/L4.png.

L5. Palabras por letra: métodos

Algoritmos.Agrupar

Una lista conserva todas las palabras. Un SortedDictionary<string, List<string>> usa la letra inicial como clave y una lista de palabras como valor.

Agrupar(palabras)

Recorre las entradas, elimina espacios exteriores y exige que todos sus caracteres sean letras. EnumerateRunes permite trabajar con puntos de código Unicode, en lugar de separar a la mitad caracteres que ocupen más de una unidad UTF-16.

Inicial y ContainsKey

Obtiene el primer Rune y lo convierte a mayúscula. Así Fresa y fresca se clasifican en F. Si esa inicial todavía no existe, crea una lista vacía; después añade la palabra al grupo.

Coordinación y salida

Estado valida primero la nueva palabra usando Agrupar con una entrada. Si es válida, la añade al historial. Vista vuelve a agrupar el historial completo. El diccionario ordena las claves; no ordena alfabéticamente las palabras dentro de cada lista.

Fragmento real que conviene explicar

```
string inicial = p.EnumerateRunes().First()
    .ToString().ToUpperInvariant();
if (!grupos.ContainsKey(inicial)) grupos[inicial] = [];
grupos[inicial].Add(p);
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 87.

L5. Recorrido y ejemplo para exponer

1. Se captura una sola palabra, por ejemplo Fresa. El controlador valida antes de modificar la lista original.

2. El método identifica la inicial F. Como todavía no hay grupo F, crea una List<string> para esa clave.

3. Al agregar Fresca se encuentra la clave F existente y se reutiliza su lista. Al agregar Oso se crea una segunda lista bajo O.

4. La interfaz muestra la lista original y, debajo, todos los grupos que devuelve el diccionario. No hay una cantidad fija de iniciales programadas.

Estado de los datos durante la ejecución

Entrada	Clave	Contenido del grupo
Fresa	F	Fresa
Fresca	F	Fresa, Fresca
Oso	O	Oso
Orilla	O	Oso, Orilla

Validaciones y límites

La cadena vacía, una frase con espacios interiores, un número o signos se rechazan. Los acentos se conservan; la versión no elimina tildes para fusionar iniciales. Las palabras repetidas se conservan como capturas independientes.

Cómo explicarlo con nuestras palabras

La clave del diccionario nos dice qué lista utilizar. Si la inicial es nueva, creamos una lista; si ya existe, añadimos la palabra a la lista existente. Así el ejercicio se adapta a nuevas letras.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/L5.png.

P1. Invertir palabra: métodos

Algoritmos.Invertir

Una Stack<Rune> almacena los caracteres. La pila trabaja como LIFO: el último elemento que entra es el primero que sale.

Invertir(entrada)

Valida y limpia el texto. El constructor de Stack recibe los Rune en el orden de lectura y los coloca en la pila; por eso el último carácter queda en la cima.

Pop y StringBuilder

Mientras Count sea mayor que cero, Pop saca el elemento de la cima. StringBuilder.Append lo agrega al resultado sin construir una cadena nueva completa en cada vuelta. Cuando la pila queda vacía, ToString entrega el texto final.

Qué se conserva

Se conserva cada punto de código Unicode completo. Esto evita romper un carácter como un emoji compuesto por un par sustituto, aunque no pretende tratar todas las secuencias combinadas de varios Rune como una sola unidad visual.

Fragmento real que conviene explicar

```
Stack<Rune> pila = new(
    Validar.Texto(entrada).EnumerateRunes());
StringBuilder r = new();
while (pila.Count > 0) r.Append(pila.Pop().ToString());
return r.ToString();
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 100.

P1. Recorrido y ejemplo para exponer

1. Se lee UTM y se valida que no sea una entrada vacía.

2. Los caracteres se apilan como U, después T y finalmente M. La cima queda en M.

3. El while desapila M y lo añade al resultado; luego hace lo mismo con T y con U.

4. La pila termina vacía y el resultado MTU se devuelve al controlador y a la pantalla. El método sólo utiliza la pila temporal, no cambia las listas de otros ejercicios.

Estado de los datos durante la ejecución

Extracción	Pila: cima a base	Resultado
Antes de extraer	M, T, U	vacío
Pop de M	T, U	M
Pop de T	U	MT
Pop de U	vacía	MTU

Validaciones y límites

Una entrada vacía o formada únicamente por espacios produce un error. Validar.Texto recorta espacios al inicio y al final; los espacios interiores forman parte del texto invertido.

Cómo explicarlo con nuestras palabras

No intercambiamos posiciones directamente en la cadena. Aprovechamos el comportamiento LIFO de la pila: lo último que leímos será lo primero que mostraremos.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/P1.png.

P2. Palíndromo: métodos

Algoritmos.Palindromo e Invertir

La cadena normalizada se compara con una segunda cadena obtenida por inversión mediante Stack<Rune>. No se usa una lista para simular la pila.

Normalización

Validar.Texto comprueba la entrada. ToLowerInvariant elimina diferencias entre mayúsculas y minúsculas. Where(Rune.IsLetterOrDigit) conserva sólo letras y dígitos, quitando espacios y signos. string.Concat une los elementos filtrados.

Comparación

Si la cadena normalizada queda vacía, se rechaza. Si contiene datos, se pasa a Invertir y se compara con ==. El resultado bool se convierte en un mensaje afirmativo o negativo dentro de Estado.Ejecutar.

Decisión sobre acentos

Los acentos no se eliminan. La aplicación no presupone que una vocal acentuada sea igual a la misma vocal sin acento. Esta regla evita cambios implícitos en el texto y está documentada como una decisión de la implementación.

Fragmento real que conviene explicar

```
if (normal.Length == 0)
    throw new ArgumentException(
        "Introduce al menos una letra o número.");
return normal == Invertir(normal);
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 107, 100.

P2. Recorrido y ejemplo para exponer

1. El usuario escribe Anita lava la tina. La comparación no se hace todavía sobre la frase original.

2. La normalización produce anitalavalatina: todas las letras quedan en minúsculas y los espacios desaparecen.

3. Invertir construye la secuencia en orden contrario usando una pila. Ambas cadenas resultan iguales.

4. Palindromo devuelve true y el controlador genera Sí es palíndromo. Con Casa se compara casa contra asac y se devuelve false.

Estado de los datos durante la ejecución

Entrada	Texto normalizado	Resultado
Anita lava la tina	anitalavalatina	Sí
12321	12321	Sí
Casa	casa	No
!!!	vacío	Error de validación

Validaciones y límites

Los signos por sí solos no se aceptan como un palíndromo vacío. Las frases se admiten porque los espacios se eliminan antes de comparar; esa regla es distinta de la validación del ejercicio de palabras por inicial.

Cómo explicarlo con nuestras palabras

Primero definimos qué caracteres cuentan para comparar. Luego reutilizamos el método de inversión con pila. La igualdad entre el texto limpio y su inversión determina el resultado.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/P2.png.

P3. Suma de números grandes: métodos

Algoritmos.Digitos y SumarGrandes

Tres Stack<int>: una por operando y otra para el resultado. Cada elemento guarda un dígito, no el número completo; por eso el tamaño no depende del rango de int o long.

Digitos(entrada)

Exige al menos un carácter y permite únicamente los caracteres ASCII de 0 a 9. c - '0' convierte cada carácter en un entero entre cero y nueve. El constructor apila los dígitos y deja las unidades en la cima.

SumarGrandes(a, b)

Repite el ciclo mientras alguna pila tenga dígitos o quede acarreo. Si un operando ya se agotó, aporta cero. Cada suma parcial produce un dígito con n % 10 y el nuevo acarreo con la división entera n / 10.

Pila resultado

Cada dígito calculado se apila. Como se calculan primero las unidades, al enumerar la pila terminada se obtiene el orden correcto. TrimStart quita ceros iniciales; si no queda ningún carácter, se devuelve la cadena 0. BigInteger se utiliza para pruebas, no para resolver esta suma.

Fragmento real que conviene explicar

```
while (p.Count > 0 || q.Count > 0 || acarreo > 0)
{
    int n = acarreo + (p.Count > 0 ? p.Pop() : 0)
        + (q.Count > 0 ? q.Pop() : 0);
    resultado.Push(n % 10);
    acarreo = n / 10;
}
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 113, 119.

P3. Recorrido y ejemplo para exponer

1. Con 592 y 3784, las cimas iniciales son 2 y 4. Los operandos se conservan como cadenas y como pilas de dígitos, sin convertirlos a un entero grande del lenguaje.

2. Se extrae un dígito de cada pila, se suma junto con el acarreo y se guarda sólo la cifra de las unidades en la pila resultado.

3. Al agotarse un operando se sigue trabajando con el otro. Si después de los últimos dígitos queda un acarreo, el while hace una vuelta adicional.

4. Al finalizar se obtiene 4376. El mismo procedimiento permite sumar miles de dígitos; el límite práctico es la memoria y el límite de longitud del formulario, no el tipo numérico de un operando.

Estado de los datos durante la ejecución

Columna	Suma parcial	Dígito	Acarreo
Unidades	2 + 4 + 0 = 6	6	0
Decenas	9 + 8 + 0 = 17	7	1
Centenas	5 + 7 + 1 = 13	3	1
Millares	0 + 3 + 1 = 4	4	0

Validaciones y límites

Se rechazan negativos, decimales, signos y letras. Los ceros iniciales sí se aceptan. La interfaz limita cada campo a 20000 caracteres: que el algoritmo no use un entero fijo no significa que el formulario sea ilimitado.

Cómo explicarlo con nuestras palabras

Hacemos la misma suma por columnas que en papel, pero con pilas. El acarreo es la parte que pasa a la siguiente columna. Nunca intentamos meter el operando completo en un int.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/P3.png.

P4. Reemplazar valor: métodos

Algoritmos.Reemplazar

Se usa la pila recibida y una Stack<int> auxiliar. La auxiliar permite revisar todos los elementos y después recuperar el orden original.

Reemplazar(pila, nuevo, viejo)

Mientras la pila original tenga elementos, extrae su cima con Pop. Si el entero coincide con viejo, cambia la variable local a nuevo y aumenta el contador cambios. Después lo apila en auxiliar.

Segundo recorrido

El primer traslado invierte el orden. El segundo while desapila la auxiliar y vuelve a apilar en la original, invirtiendo una segunda vez. Dos inversiones restauran el orden relativo de los elementos.

Valor devuelto y referencias

El método devuelve la cantidad de sustituciones y modifica la misma pila recibida. No devuelve una pila nueva: Stack es un objeto de referencia. El controlador convierte la pila final a un arreglo para mostrarla de cima a base.

Fragmento real que conviene explicar

```
while (pila.Count > 0)
{
    int n = pila.Pop();
    if (n == viejo) { n = nuevo; cambios++; }
    auxiliar.Push(n);
}
while (auxiliar.Count > 0) pila.Push(auxiliar.Pop());
return cambios;
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 127.

P4. Recorrido y ejemplo para exponer

1. La entrada 1,2,3,2 se separa por comas y cada elemento se convierte a entero. Se apilan de izquierda a derecha, de modo que el último 2 queda en la cima.

2. Con viejo 2 y nuevo 9, cada aparición se sustituye al salir. Los elementos que no coinciden se conservan.

3. La pila auxiliar queda con el orden invertido. El segundo while la vacía hacia la original para recuperar el orden.

4. Se muestran la pila 9,3,9,1 desde la cima y el contador 2. Si el valor no aparece, el contador es cero y el orden permanece intacto.

Estado de los datos durante la ejecución

Momento	Cima a base	Cambios
Original	2, 3, 2, 1	0
Auxiliar al terminar el primer ciclo	1, 9, 3, 9	2
Original restaurada	9, 3, 9, 1	2

Validaciones y límites

Una entrada como 1,,2 contiene un elemento vacío y se rechaza. Las letras o enteros fuera del rango de int también se rechazan. El método admite una pila vacía y devuelve cero, aunque el formulario pide al menos un valor.

Cómo explicarlo con nuestras palabras

Si usáramos sólo una pila auxiliar, el resultado quedaría al revés. El segundo traslado es el que mantiene el orden y permite cambiar todas las coincidencias sin desordenar los demás valores.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/P4.png.

C1. Ventanilla de banco: métodos

Cola<T>, Cliente y Banco

Banco usa una Cola<Cliente> de capacidad limitada. Internamente Cola<T> encapsula una Queue<T>. Es FIFO: el cliente con mayor tiempo en la cola se atiende primero.

ColaLlena e InsertaCola

ColaLlena compara Count con Capacidad. InsertaCola comprueba esa condición antes de usar Enqueue, que agrega al final. La capacidad debe ser mayor que cero; el valor inicial de la pantalla es cinco.

Llegar(nombre, movimiento, ahora)

Comprueba el cupo, valida nombre y movimiento y crea un Cliente con turno + 1 y la hora recibida. Después de insertar incrementa turno. De esta forma una inserción fallida no consume un número de turno.

Atender(ahora)

EliminaCola rechaza la cola vacía; si hay clientes, Dequeue retira el frente. La espera se calcula restando Llegada a la hora de atención y tomando segundos completos. Frente usa Peek sin eliminar y Final obtiene el último elemento.

Fragmento real que conviene explicar

```
public (Cliente Cliente, long Segundos)
    Atender(DateTime ahora)
{
    var c = Clientes.EliminaCola();
    return (c, Math.Max(0,
        (long)(ahora - c.Llegada).TotalSeconds));
}
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 135, 149, 153.

C1. Recorrido y ejemplo para exponer

1. El usuario elige el movimiento en el selector y escribe el nombre. Estado sólo permite uno de los movimientos ofrecidos por la pantalla.

2. Al agregar, el cliente recibe un turno y DateTime.Now como hora de llegada. La tabla se actualiza y el cliente se forma al final.

3. Atender retira automáticamente el primer cliente; no pide un turno para elegir a alguien intermedio. Se muestran su nombre y su espera.

4. Después de cada operación se actualizan frente y final. Si la cola queda vacía, la pantalla usa -1 como indicador. La capacidad sólo se puede cambiar con la cola vacía.

Estado de los datos durante la ejecución

Evento	Cola: frente a final	Efecto
Ana llega a las 12:00:00	Ana, turno 1	Frente y final = 1
Luis llega a las 12:00:02	Ana, Luis	Frente 1; final 2
Atender a las 12:00:18	Luis	Ana esperó 20 segundos

Validaciones y límites

La cola llena impide nuevas inserciones y la vacía impide atender. Se rechazan nombres faltantes, movimientos ajenos al selector y cambios de capacidad con clientes pendientes. Math.Max evita mostrar tiempos negativos.

Cómo explicarlo con nuestras palabras

La cola impone el orden de atención. Agregar utiliza el final y atender utiliza el frente. La espera se calcula con las horas guardadas, no con una cantidad inventada por la interfaz.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/C1.png.

C2. Juego de pintar coches: métodos

ColorAuto, Coche y Juego

Juego mantiene una Cola<Coche> con capacidad cinco. Cada coche tiene un número y un color solicitado. Las variables de tiempo se expresan en milisegundos y se basan en Environment.TickCount64.

Iniciar y Encolar

Iniciar vacía la cola anterior, pone el marcador en cero, conserva el récord y crea el primer coche inmediatamente. Encolar elige aleatoriamente uno de los cinco colores y aumenta la secuencia. La siguiente llegada se programa como ahora + Intervalo.

Actualizar(ahora)

Mientras la partida siga activa y haya vencido la siguiente llegada, agrega coches. El while recupera llegadas vencidas si la pestaña se retrasó. Cuando la cola alcanza cinco, marca Activo = false y guarda el instante exacto de fin.

Pintar e Intervalo

Pintar primero actualiza las llegadas y valida partida, cola y color del frente. Sólo un acierto elimina ese frente e incrementa Pintados. Intervalo calcula max(2000, 20000 - Pintados * 2000): cada acierto reduce dos segundos hasta un mínimo de dos.

Récord y visualización

Transcurrido usa el reloj actual si la partida sigue activa o Fin si terminó. GuardarRecord actualiza puntaje y tiempo; motor.js conserva estos datos en localStorage para la versión pública. La pantalla consulta el estado cada segundo cuando este ejercicio está seleccionado.

Fragmento real que conviene explicar

```
Coches.EliminaCola(); Pintados++;
ProximaLlegada = Math.Min(ProximaLlegada,
    ahora + Intervalo);
GuardarRecord(ahora);
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 202, 213, 227.

C2. Recorrido y ejemplo para exponer

1. Al iniciar aparece un coche. Si nadie pinta, llegan otros a los 20, 40, 60 y 80 segundos; al quinto coche termina la partida.

2. Al acertar el color, el frente sale y el marcador aumenta. Si era el primer acierto, Intervalo cambia de 20000 a 18000 milisegundos.

3. Math.Min conserva la llegada que ocurra antes: la ya programada o ahora + el intervalo reducido. Acertar nunca retrasa una llegada que ya estaba próxima.

4. El segundo acierto reduce a 16 segundos, el tercero a 14 y así hasta dos. Un error de color conserva el coche y el marcador. Si una llegada acaba de llenar la cola antes de pintar, ya no se admite el acierto.

Estado de los datos durante la ejecución

Coches pintados	Cálculo en milisegundos	Intervalo
0	20000 - 0	20 segundos
1	20000 - 2000	20 segundos
3	20000 - 6000	14 segundos
9 o más	mínimo permitido: 2000	2 segundos

Validaciones y límites

No se puede pintar sin partida, con la cola vacía ni después del final. Se comprueba únicamente el color del frente. Cambiar de ejercicio no pausa el reloj: al consultar otra vez se calculan las llegadas que hayan vencido.

Cómo explicarlo con nuestras palabras

La dificultad no aumenta reduciendo un contador decorativo: cambia el intervalo real de llegada. Cada acierto resta dos segundos y además puede adelantar la siguiente llegada. El límite inferior evita intervalos cero o negativos.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/C2.png.

C3. Estacionamiento de autos: métodos

Auto, Recibo y ColaCircularDoblementeLigada

Esta cola no se delega a Queue<T>. Cada Nodo contiene un Auto y dos referencias: Anterior y Siguiente. El último apunta al primero y el primero al último. frente identifica al siguiente vehículo que debe salir.

Insertar(placas, propietario, ahora)

Normaliza las placas a mayúsculas, busca duplicados y valida al propietario. Si es el primer nodo, sus dos enlaces apuntan a él mismo. En otro caso inserta después del último, que se obtiene como frente.Anterior, sin cambiar el frente.

Salir(ahora)

Si no hay frente, informa que está vacío. Con un solo nodo pone frente en null. Con varios avanza frente al siguiente y une el último con el nuevo frente en ambos sentidos. Reduce Count y construye un Recibo con horas, segundos y costo.

Lista y EnlacesValidos

Lista usa do/while para recorrer una vuelta y se detiene cuando vuelve al frente. EnlacesValidos comprueba que Siguiente.Anterior y Anterior.Siguiente regresen al mismo nodo y que se visite exactamente Count nodos. Ayuda a detectar enlaces mal actualizados en las pruebas.

Fragmento real que conviene explicar

```
if (frente is null)
{
    n.Anterior = n.Siguiente = n; frente = n;
}
else
{
    Nodo ultimo = frente.Anterior;
    n.Anterior = ultimo; n.Siguiente = frente;
    ultimo.Siguiente = n; frente.Anterior = n;
}
Count++;
```

Fuente: CSharp/Modelos.cs. Líneas de inicio de las declaraciones relacionadas: 164, 175, 184, 192.

C3. Recorrido y ejemplo para exponer

1. Se registra ABC-123 con su propietario y la hora del sistema. El primer nodo se enlaza consigo mismo; Count pasa a uno.

2. Al registrar XYZ-987 se conecta detrás de ABC-123. El circuito queda cerrado en ambos sentidos y el frente continúa en ABC-123.

3. Salida de auto retira ABC-123 por FIFO. XYZ-987 pasa a ser el frente y, al quedar solo, sus dos enlaces vuelven a apuntar a él mismo.

4. Se calcula la diferencia entre salida y entrada, se toman segundos completos y se multiplican por 2m. Por ejemplo, 30 segundos cuestan $60.00. La cola no tiene un cupo fijo programado; su límite práctico es la memoria.

Estado de los datos durante la ejecución

Situación	Enlace siguiente	Enlace anterior
Un nodo A	A apunta a A	A apunta a A
Dos nodos A y B	A a B; B a A	A a B; B a A
Sale A; queda B	B apunta a B	B apunta a B
Sale B	frente = null	Count = 0

Validaciones y límites

Se rechazan placas repetidas aunque cambien las mayúsculas, textos vacíos y salida sin autos. La salida es siempre del frente, no por placas. La tarifa usa segundos completos y evita cobros negativos si el reloj retrocede.

Cómo explicarlo con nuestras palabras

El punto importante son los enlaces. Siempre debemos reparar el enlace de ida y el de regreso. El caso de un solo auto se trata aparte para dejar la cola realmente vacía al retirarlo.

Evidencia relacionada: la captura de este ejercicio aparece en el desarrollo inicial y en evidencias-web/publica/C3.png.

Juego: entender los tiempos y la dificultad

```
public long Intervalo =>
    Math.Max(2000, 20000 - Pintados * 2000L);
```

Los valores están en milisegundos: 1000 milisegundos equivalen a un segundo. 20000 representa 20 segundos y 2000 representa dos. La L al final del literal numérico hace explícito el tipo long. Math.Max impide que al acumular muchos aciertos el intervalo llegue a cero o sea negativo.

Por qué usamos Math.Min para la siguiente llegada

Ejemplo	Cálculo	Efecto
Primer acierto a los 0.1 segundos	min(20, 0.1 + 18) = 18.1	La primera llegada pendiente se adelanta a 18.1 segundos.
Primer acierto a los 3 segundos	min(20, 3 + 18) = 20	Se mantiene la llegada ya prevista a los 20 segundos.
La pestaña estuvo detenida por el navegador	Actualizar revisa todas las llegadas vencidas	El while agrega los coches pendientes hasta ponerse al día o terminar la partida.

Reducir el intervalo no significa conceder siempre un intervalo completo nuevo desde el clic: si ya había una llegada más cercana, ésta se conserva. Esta decisión hace que un acierto nunca retrase la presión de la cola.

Partida y récord son datos diferentes

Pintados pertenece a la partida actual y se reinicia al comenzar. Record y TiempoRecord se conservan. GuardarRecord los actualiza al superar el puntaje anterior y, al terminar con ese mismo puntaje, registra el tiempo hasta el fin. No se implementa una clasificación global de jugadores ni un criterio de menor tiempo para desempatar.

En la web pública, motor.js recuerda esos dos valores con localStorage y los restaura al arrancar. No guarda los coches ni las listas completas. Al recargar la página se inicia un estado nuevo para los ejercicios, con el récord recuperado si el navegador permite almacenamiento.

Variante ASP.NET: rutas y sesiones

La copia local incluida en Publicacion no usa el motor WebAssembly para las operaciones: ejecuta una aplicación ASP.NET Core. La misma interfaz envía solicitudes HTTP al servidor local. Los modelos y reglas son los mismos, pero cambia el lugar de ejecución.

Componente	Funcionamiento
GET /api/estado	Obtiene la sesión y devuelve Vista. La interfaz lo utiliza al iniciar y al actualizar el juego. No realiza una inserción solicitada por el usuario, aunque Vista sí actualiza las llegadas del juego según el reloj.
POST /api/accion	Recibe la Solicitud, exige X-Practica = 1 y valida el tamaño de los valores. Llama a Estado.Ejecutar y responde con mensaje y estado. Los errores de validación devuelven HTTP 400.
Sesion(context, cache)	Busca un identificador de cookie válido. Si no existe un Estado asociado, crea un identificador aleatorio y un Estado nuevo. No comparte las listas entre todos los navegadores.
Cookie y MemoryCache	La cookie es HttpOnly y SameSite Strict; Secure se usa cuando la solicitud es HTTPS. La caché conserva hasta 1000 sesiones, con expiración deslizante de dos horas de inactividad. No es una base de datos.
lock(estado)	Evita que dos solicitudes modifiquen el mismo Estado simultáneamente. Vista toma copias de las colecciones mutables para preparar la respuesta.

Diferencias que hay que explicar correctamente

En GitHub Pages los modelos viven en el navegador de cada usuario. En ASP.NET viven en el proceso del servidor. En la primera variante las horas se obtienen en el dispositivo del navegador; en la segunda se obtienen en el servidor. Cerrar o reiniciar un servidor elimina sus estados en memoria.

Abrir INICIAR_LOCAL.cmd inicia el ejecutable portable con su carpeta Publicacion como directorio de trabajo, espera a que responda y abre el navegador. La ventana de control permite detener esa instancia pulsando Enter.

Qué comprobamos y qué demuestra cada prueba

La documentación usa los resultados guardados de las ejecuciones reales, no una lista de pruebas inventada. No es necesario volver a compilar el programa para cambiar la portada o ampliar esta explicación.

Evidencia	Qué verifica
pruebas-web.txt	22 verificaciones HTTP: operaciones de los doce ejercicios, valores límite, productos y placas, sobreflujo del banco, entrada inválida, aislamiento de sesiones y cabecera de solicitudes. Incluye el intervalo de 20 segundos tras el primer acierto.
pruebas-publica.txt	Los doce ejercicios se ejecutaron desde sus controles en el navegador con C# WebAssembly. Se confirmó también la reducción de 20 a 20 segundos.
evidencias-web/publica	Imágenes capturadas después de realizar las operaciones. El código del modelo produce los resultados; las imágenes permiten comprobar cómo se presentan al usuario.
CSharp/Pruebas.cs	Suite de modelos e interfaz de escritorio. Incluye sumas comparadas con BigInteger como oráculo, FIFO, tiempos controlados y comprobación de enlaces al insertar y retirar nodos. Su registro histórico de escritorio no sustituye las pruebas web.

Casos útiles para demostrar en clase

• Calificación 7 aprueba y 6.99 reprueba, sin redondear la comparación.

• Sumar una cadena de muchos nueves y uno produce un uno seguido de ceros; muestra el acarreo final.

• Agregar dos clientes y atender una vez retira al primero, no al que se escribió más recientemente.

• Pintar con el color equivocado mantiene la cola y el marcador. Un acierto cambia el intervalo.

• Insertar dos autos y sacar ambos comprueba el caso de varios nodos y el caso especial del último nodo.

Probar no significa únicamente que la pantalla abra: también debemos comprobar que los datos cambian como se espera y que los errores no dejan la estructura en un estado incorrecto.

Guion práctico para la exposición del equipo

Introducción común

Esta práctica resuelve doce problemas mediante listas, pilas y colas. Separamos los datos y los algoritmos de los formularios. La versión pública ejecuta el código C# en el navegador y la alternativa local utiliza ASP.NET Core.

Orden para presentar cualquier ejercicio

1. Explicar el problema en una frase y mencionar los datos que se capturan.
2. Identificar la clase y la estructura utilizada: List, Stack, Queue o nodos enlazados.
3. Abrir el método principal y describir el orden de las instrucciones. Indicar qué regla decide el resultado.
4. Mostrar un ejemplo pequeño y decir cómo cambian los datos, no sólo leer el mensaje final.
5. Probar un caso inválido y explicar qué validación lo detiene.
6. Cerrar explicando por qué la estructura elegida ayuda a resolver ese problema.

Reparto sugerido, ajustable por el equipo

Integrante	Ejercicios para preparar
Santiago Nicolás De la Mora Núñez	L1, L2 y L3: productos, paridad y alumnos.
Jesus Joel Canche Chac	L4, L5 y P1: compra, grupos por inicial e inversión.
Roger Alejandro Aguilar Nuñez	P2, P3 y P4: palíndromos, suma y reemplazo.
Víctor Manuel Almendarez García	C1, C2 y C3: banco, juego y estacionamiento.

Es una propuesta de organización, no un registro de quién programó cada módulo. Todos deben comprender el recorrido general y la diferencia entre las dos variantes web.

Antes de exponer

Tener abierta la web, ubicar Modelos.cs y el método correspondiente, elegir datos pequeños y ensayar el caso de error. En el juego no hace falta esperar muchos coches para demostrar la dificultad: el primer acierto ya cambia el intervalo de 20 a 20 segundos.

Preguntas frecuentes y vocabulario

¿Por qué no sumar los operandos grandes con int?

Porque una cadena con cientos o miles de dígitos excede su rango. La solución almacena dígitos individuales en pilas y maneja un acarreo pequeño.

¿Por qué dos pasadas en Reemplazar?

La primera revisa y modifica mientras invierte el orden. La segunda restaura ese orden; por eso la cima final sigue representando la cima original con los valores sustituidos.

¿Por qué el estacionamiento no usa simplemente Queue?

El ejercicio exige una cola circular doblemente ligada. Los nodos, sus dos enlaces y los casos de inserción y eliminación están programados explícitamente.

¿El tiempo de espera y el costo son aleatorios?

No. Se calculan restando las horas guardadas a la hora actual. Sólo cantidades y precios de productos, números de paridad y colores del juego se generan aleatoriamente.

¿Qué es una instancia?

Es un objeto concreto creado a partir de una clase. Por ejemplo, cada Estado contiene su propia instancia de Banco, con sus clientes y su contador de turnos.

¿Qué significa genérico, como Cola<T>?

T representa el tipo de elemento. La misma clase puede almacenar Cliente o Coche conservando las operaciones de cola y las comprobaciones de capacidad.

¿Qué es un nodo?

Es el objeto que reúne un dato y referencias a otros nodos. En el estacionamiento incluye un Auto y los enlaces Anterior y Siguiente.

¿Qué parte conserva datos después de recargar?

En la versión pública sólo el récord y su tiempo se guardan con localStorage. Las listas y las colas se reinician; no hay una base de datos persistente de alumnos, productos o clientes.

Mapa de métodos en el código entregado

Estas referencias indican la línea donde comienza una declaración en CSharp/Modelos.cs. No son números de página del informe ni una lista de todos los archivos. Permiten encontrar rápidamente la clase o el método al explicar el programa.

Ejercicio	Clase o métodos principales	Líneas de inicio
L1	Producto y Almacen	25, 32, 39
L2	Algoritmos.Clasificar	80
L3	Alumno y Grupo	26, 48
L4	Compra y Producto	60, 75, 76
L5	Algoritmos.Agrupar	87
P1	Algoritmos.Invertir	100
P2	Algoritmos.Palindromo e Invertir	107, 100
P3	Algoritmos.Digitos y SumarGrandes	113, 119
P4	Algoritmos.Reemplazar	127
C1	Cola<T>, Cliente y Banco	135, 149, 153
C2	ColorAuto, Coche y Juego	202, 213, 227
C3	Auto, Recibo y ColaCircularDoblementeLigada	164, 175, 184, 192

Archivo	Declaración	Línea
Web/Estado.cs	Ejecutar(Solicitud s)	16
Web/Estado.cs	Vista()	56
WebPublica/Program.cs	MotorWeb	9
Web/wwwroot/app.js	act(action, label)	70
Web/wwwroot/app.js	render()	40

Los números se calculan directamente de los archivos al regenerar el informe. Si se agregan líneas al programa en el futuro, hay que regenerar también esta guía. La documentación describe el comportamiento del código entregado y las decisiones actuales; no atribuye persistencia ni capacidades que la aplicación no implementa.